

Open-Loop Motion in ROS 2

Four Primitives, One Timer, and a Square That Does Not Close

Prof. Anis Koubaa Dr. Asem Ibrahim Alalwan

EE 414 — Introduction to Robotics
College of Engineering
Alfaisal University

Fall 2026 — Week 4, Lecture B



Lecture Overview

Roadmap

Session A gave you v , ω and the model. This session turns them into running code: command motion **from the terminal**, then write the four primitives — **straight**, **rotate**, **arc**, **spiral** — as an rclpy node, compose them into a square, and run the identical node against TurtleBot 3 in Gazebo Harmonic. The square closes on the turtle. It will not close on the robot.

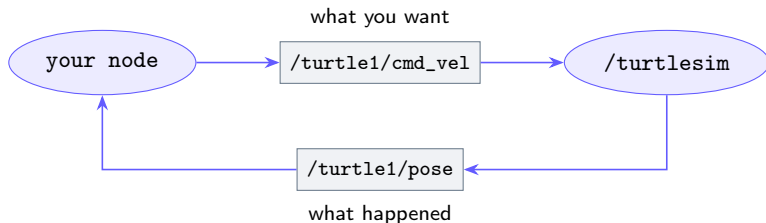
- 1 Commanding Motion
- 2 The Four Primitives
- 3 A Motion Node in rclpy
- 4 Composing Primitives
- 5 The Same Node on a Real Robot
- 6 Wrap-Up

Part 1

Commanding Motion

- 1 Commanding Motion
- 2 The Four Primitives
- 3 A Motion Node in rclpy
- 4 Composing Primitives
- 5 The Same Node on a Real Robot
- 6 Wrap-Up

The Shape of Every Motion Program



Two topics and nothing else. Everything in this session is a decision about *what to put on the top arrow, and when to change it*.

The bottom arrow, this week

We subscribe to the pose to **measure** what happened, print it, and compare it against what was asked — and then we do nothing about it. Closing that loop is next week's subject. Today it is instrumentation, not control.

The Two Topics, and Their Types

Topic	Type	Direction
/turtle1/cmd_vel	geometry_msgs/msg/Twist	you → simulator
/turtle1/pose	turtlesim/msg/Pose	simulator → you

the two fields you will set

```
msg.linear.x    = v  
msg.angular.z   = w
```

the three you will read

```
msg.x  msg.y  msg.theta
```

Two fields out, three fields in. That is the entire interface to a moving robot.

Motion Before Code: the Terminal

```
$ ros2 run turtlesim turtlesim_node

$ ros2 topic pub -r 10 /turtle1/cmd_vel geometry_msgs/msg/Twist \
  '{linear: {x: 2.0}, angular: {z: 1.8}}'
```

Two terminals, no package, no build. The turtle drives a circle and leaves a trail — and the trail is the arc from session A, radius $R = v/\omega = 2.0/1.8 = 1.11$.

In the field

Do this before writing a node, every time you meet a new robot. If the terminal cannot move it, your code will not either, and you will have spent an hour debugging the wrong file. It is also how you discover the topic is called `/cmd_vel_unstamped` on the robot you were given.

Why `-r 10`: a Stream, Not a Command

`ros2 topic pub` without `-r` sends the message once. The turtle twitches and stops.

 publishing at 10 Hz: the turtle moves

 publishing stops: **the turtle stops**

A velocity command is a **standing order with an expiry**. The driver keeps the last value only briefly; when the stream stops, the movement stops.

This is a safety property, not an inconvenience

If your node crashes mid-drive, a robot that held the last command would keep going into a wall. Because velocity must be refreshed, a dead publisher is a stopped robot. Every mobile base you will ever use behaves this way.

Choosing the Rate

Rate	Δt	Consequence
1 Hz	1.0 s	Visibly jerky. Each command lasts a whole second before the next correction can arrive.
10 Hz	0.1 s	The default for this course. Smooth enough to watch, coarse enough to reason about.
50 Hz	0.02 s	What a real base runs internally. More than a Python control node needs.

The rate is not a cosmetic choice: $\Delta t = 1/\text{rate}$ is the same Δt that appeared in session A's integration. Your open-loop timing is only as accurate as your timer's period.

Pick the rate once, write it down as one constant, and use it for both the publisher and the arithmetic.

Part 2

The Four Primitives

- 1 Commanding Motion
- 2 The Four Primitives
- 3 A Motion Node in rclpy
- 4 Composing Primitives
- 5 The Same Node on a Real Robot
- 6 Wrap-Up

Four Patterns, One Message

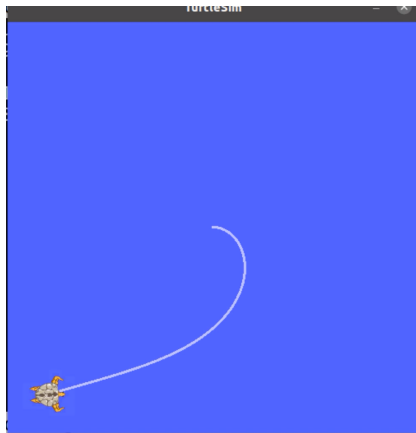
Primitive	linear.x	angular.z	Path
Straight line	v (constant)	0	a line, heading unchanged
Rotate in place	0	ω	a point, position unchanged
Arc	v	ω	circle of radius $R = v/\omega$
Spiral	$f(t)$, rising	ω (const)	radius grows with every turn

There is no fifth pattern. Every trajectory in this course is one of these four, or a sequence of them, or a controller choosing between them at 10 Hz.

In the field

This table is the reason `Twist` has only two useful fields for a diff-drive: with two numbers you have already exhausted what the mechanism can do. Everything else is scheduling.

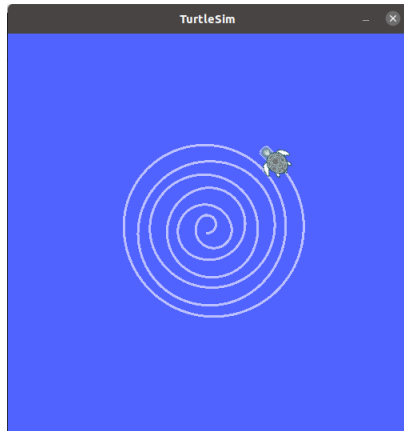
Primitive 3: Both Fields Non-Zero



The turtle was given one `Twist` and it was never changed. **The curve is not in the program** — it is what v and ω held together mean.

Capture from A. Koubaa, *ROS for Beginners: Basics, Motion and OpenCV* (2020). The turtlesim window is unchanged under ROS 2 Jazzy.

Primitive 4: One Field Changing with Time



ω constant, v rising a little on every tick. The radius v/ω grows with it, and the arc never closes. **Nine lines of code produce this.**

Capture from A. Koubaa, *ROS for Beginners: Basics, Motion and OpenCV* (2020).

Divide and Conquer

- ➊ A function that moves in a **straight line** for a given distance, forward and backward.
- ➋ A function that **rotates in place** by a given angle, clockwise and counter-clockwise.
- ➌ Compose the two into a **square**, and measure how far from home it finishes.
- ➍ A function that draws a **spiral**.

Each one is a few lines. Each one is testable on its own, in a window you can watch. The square is the first thing in this course that is built out of parts you wrote yourself.

Write the primitives, then compose. A single `drive_a_square()` function that does everything cannot be debugged, because nothing inside it can be run alone.

Part 3

A Motion Node in rclpy

- 1 Commanding Motion
- 2 The Four Primitives
- 3 A Motion Node in rclpy
- 4 Composing Primitives
- 5 The Same Node on a Real Robot
- 6 Wrap-Up

The Skeleton

```
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Twist

class Mover(Node):
    def __init__(self):
        super().__init__('mover')
        self.pub = self.create_publisher(Twist, '/turtle1/cmd_vel', 10)
        self.timer = self.create_timer(0.1, self.tick) # 10 Hz, forever
        self.elapsed = 0.0

    def tick(self):
        pass
```

Everything on this slide was Week 2 and Week 3: a node, a publisher, a timer. Only `tick` is new, and it is the only thing that changes between the four primitives.

Open-Loop Timing: One Division

To cover d metres at speed v , publish that `Twist` for

$$t = \frac{d}{v} \text{ seconds,} \quad \text{then publish zero.}$$

To turn ϕ radians at rate ω , publish for $t = \phi/\omega$, then publish zero.

Goal	v or ω	Time	Ticks at 10 Hz
2.0 forward	$v = 1.0$	2.0 s	20
90° left	$\omega = 0.5$	3.14 s	31.4

31.4 ticks is not a whole number. You will get 31 or 32, and the difference is 0.9° of heading you did not ask for — on *every* corner.

The first error in an open-loop program is not slip or friction. It is integer division.

move_straight

```
class MoveStraight(Node):
    def __init__(self, speed=1.0, distance=2.0):
        super().__init__('move_straight')
        self.pub = self.create_publisher(Twist, '/turtle1/cmd_vel', 10)
        self.dt = 0.1
        self.speed = speed
        self.duration = abs(distance) / abs(speed)
        self.elapsed = 0.0
        self.timer = self.create_timer(self.dt, self.tick)

    def tick(self):
        msg = Twist()
        msg.linear.x = self.speed if self.elapsed < self.duration else 0.0
        self.pub.publish(msg)
        self.elapsed += self.dt
```

Backwards is a negative speed; abs keeps the duration positive.

Why a Timer and Not sleep

What a beginner writes

```
while elapsed < duration:  
    pub.publish(msg)  
    time.sleep(0.1)  
    elapsed += 0.1
```

What works

```
self.create_timer(0.1, self.tick)  
rclpy.spin(node)
```

The loop on the left publishes correctly and is deaf: while it sleeps, `spin` is not running, so no pose arrives. It works today and blocks you the moment you add the subscriber two slides from now.

The rule from Week 3, restated

`spin` runs one callback at a time. Anything that blocks inside a callback — `sleep`, a synchronous service call, a long computation — blocks every other callback in the node. Put the waiting in the timer's period, never in the callback body.

rotate_in_place

```
from math import radians

class Rotate(Node):
    def __init__(self, deg_per_s=30.0, angle_deg=90.0, clockwise=False):
        super().__init__('rotate')
        self.pub = self.create_publisher(Twist, '/turtle1/cmd_vel', 10)
        self.dt = 0.1
        self.w = radians(deg_per_s) * (-1.0 if clockwise else 1.0)
        self.duration = radians(angle_deg) / abs(self.w)
        self.elapsed = 0.0
        self.timer = self.create_timer(self.dt, self.tick)
```

tick is move_straight's, with msg.angular.z in place of msg.linear.x. Degrees enter at the top and are converted at once; below that line the code is radians only — and **negative** angular.z is **clockwise**, because +z is counter-clockwise.

Stopping Is a Message

```
self.pub.publish(Twist())    # all six fields default to 0.0
```

A default-constructed `Twist` is a full stop. Every primitive in this session ends with one, and every node you write from now on should send one before it exits.

In the field

Ctrl-C kills the node, the stream stops, the base times out and halts — usually. A node that publishes its own stop first does not depend on “usually”. On a 30 kg platform that is a dented wall.

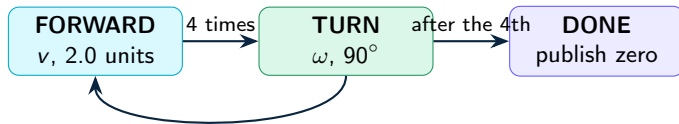
The last thing a motion node does is ask the robot to stop.

Part 4

Composing Primitives

- 1 Commanding Motion
- 2 The Four Primitives
- 3 A Motion Node in rclpy
- 4 Composing Primitives**
- 5 The Same Node on a Real Robot
- 6 Wrap-Up

The Square Is a State Machine



One timer, one variable holding which state you are in, and a counter. The timer keeps running at 10 Hz throughout; only the contents of the published Twist change.

Why not four nodes, or four functions called in a row

Because a function that drives forward and returns when finished must block — and we have just spent a slide on why a motion node must never block. A state machine on a timer is the non-blocking way to say “do this, then that”.

The Square, in the Timer

```
def tick(self):
    msg = Twist()
    if self.sides == 4:
        self.pub.publish(msg)
        return
    if self.state == 'FORWARD':
        msg.linear.x = self.v
        if self.elapsed >= self.t_forward:
            self.state, self.elapsed = 'TURN', 0.0
    else:
        msg.angular.z = self.w
        if self.elapsed >= self.t_turn:
            self.state, self.elapsed = 'FORWARD', 0.0
            self.sides += 1
    self.pub.publish(msg)
    self.elapsed += self.dt
```

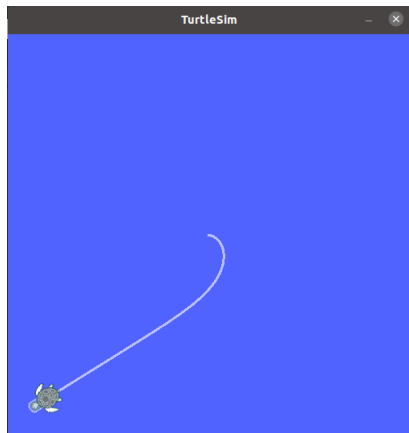
The Spiral

```
def tick(self):
    msg = Twist()
    msg.linear.x = self.a + self.b * self.elapsed    # v grows linearly
    msg.angular.z = self.w                          # w does not
    self.pub.publish(msg)
    self.elapsed += self.dt
```

Two lines of body. The radius at time t is $R(t) = (a + b t)/\omega$, growing at a constant b/ω per second — which is why the gap between successive turns is even.

A spiral is the classic coverage pattern for a cleaning robot: no gaps, no map. Set b just under the robot's width and you have a naive floor-cleaner.

The Trail Is the Evidence



The turtle started bottom-left and stopped where the trail ends. **Whether it arrived is a question you answer from /turtle1/pose, not by looking** — and this session only asks the question. Next week it acts on the answer.

Measuring, Not Correcting

```
from turtlesim.msg import Pose
self.create_subscription(Pose, '/turtle1/pose', self.on_pose, 10)

def on_pose(self, msg):
    if self.start is None:
        self.start = (msg.x, msg.y)
    self.error = hypot(msg.x - self.start[0], msg.y - self.start[1])
```

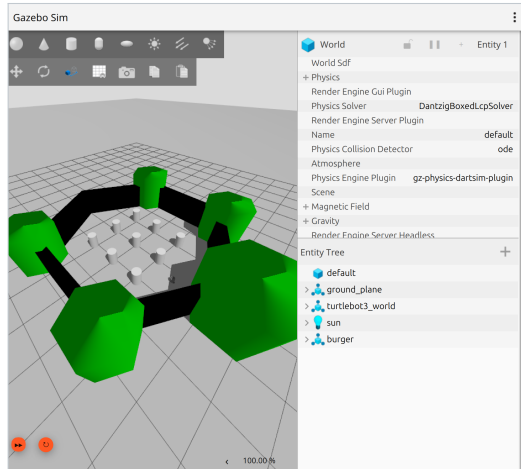
It stores the first pose it sees and, from then on, the distance back to it. When the square finishes, `self.error` is how far from home the robot ended up — and nothing in `tick` ever reads it. **This is the number you report:** nearly zero in `turtlesim`, not in `Gazebo Harmonic`, and that gap is the point of the session.

Part 5

The Same Node on a Real Robot

- 1 Commanding Motion
- 2 The Four Primitives
- 3 A Motion Node in rclpy
- 4 Composing Primitives
- 5 The Same Node on a Real Robot
- 6 Wrap-Up

Where the Rest of the Course Runs



TurtleBot 3 Burger in Gazebo Harmonic. A floor, mass, friction and wheels that can slip — everything turtlesim leaves out. **The node you just wrote runs here unchanged except for two topic names.**

Launching It

```
$ export TURTLEBOT3_MODEL=burger
$ ros2 launch turtlebot3_gazebo empty_world.launch.py

$ ros2 topic pub -r 10 /cmd_vel geometry_msgs/msg/Twist \
    '{linear: {x: 0.1}, angular: {z: 0.0}}'
```

Same command shape as the turtle, three differences: the topic has no /turtle1 prefix, the world must be launched first, and **0.1 is a sensible speed** where 2.0 was sensible for a turtle.

The environment variable is not optional

TURTLEBOT3_MODEL tells the launch file which robot description to load. Unset, the launch fails with a message about a missing model — the single most common first-run failure on this platform. Put it in your shell profile once.

What Changes, and What Does Not

	Turtlesim	TurtleBot 3 in Gazebo Harmonic
Command topic	/turtle1/cmd_vel	/cmd_vel
Command type	geometry_msgs/msg/Twist	identical
Feedback topic	/turtle1/pose	/odom
Feedback type	turtlesim/msg/Pose	nav_msgs/msg/Odometry
Heading	theta, radians	a quaternion, four floats
Sensible v	1.0–2.0 canvas units	0.10–0.22 m s ⁻¹
Slip	none, ever	yes

Five of these are renames. Two — the heading representation and the slip — are real, and they are the two that will cost you time.

Write the topic names and the speed as parameters, and one node serves both.

Reading /odom

```
from nav_msgs.msg import Odometry

self.create_subscription(Odometry, '/odom', self.on_odom, 10)

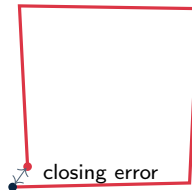
def on_odom(self, msg):
    p = msg.pose.pose.position
    q = msg.pose.pose.orientation
    yaw = atan2(2.0 * (q.w * q.z + q.x * q.y),
               1.0 - 2.0 * (q.y * q.y + q.z * q.z))
    self.pose = (p.x, p.y, yaw)
```

Longer than the turtlesim version, and it produces the same three numbers. Write it once, put it in a helper, reuse it in Weeks 5, 6, 11 and 12. `msg.pose.pose` is not a typo: `Odometry` holds a `PoseWithCovariance`, which holds a `Pose` — the covariance is the robot's own estimate of how wrong it is, unused today and essential in Week 12.

The Square That Does Not Close



turtlesim: closes



Gazebo Harmonic: does not

Identical code, identical numbers, identical timing. The turtle is a perfect integrator; the simulated robot has mass that takes time to accelerate, wheels that slip, and a 90° turn that is really 88° — four times over.

Report both closing errors. The one you cannot make small by trying harder is the reason Week 5 exists.

Part 6 Wrap-Up

- 1 Commanding Motion
- 2 The Four Primitives
- 3 A Motion Node in rclpy
- 4 Composing Primitives
- 5 The Same Node on a Real Robot
- 6 Wrap-Up

What Breaks, and Why

Symptom	Cause, and the fix
Turtle twitches and stops	Published once. A velocity needs a stream — use <code>-r</code> , or a timer.
Nothing moves at all	Wrong topic name. <code>ros2 topic list</code> and read it, do not guess the prefix.
Turns are wildly too big	Degrees passed where radians were expected. Convert at the boundary, once.
Turns the wrong way	Sign of <code>angular.z</code> . Positive is counter-clockwise, always.
Robot keeps going after Ctrl-C	No stop message was published. Send <code>Twist()</code> before exiting.
Pose callback never fires	Something blocks in the timer callback — almost always a sleep.
Gazebo launch fails	<code>TURTLEBOT3_MODEL</code> not exported in <i>this</i> terminal.

Checkpoints and Deliverables

- 1 **Terminal:** drive the turtle in a circle with `ros2 topic pub -r 10`, and say what R should be before you press Enter.
- 2 **move_straight:** a node, on a timer, that stops.
- 3 **rotate_in_place:** 90° each way, in radians.
- 4 **Square:** the state machine, with the closing error printed from `/turtle1/pose`.
- 5 **Spiral:** two lines in `tick`, and a screenshot of the trail.
- 6 **TurtleBot 3:** the same square in Gazebo Harmonic, closing error from `/odom`.

Submit both closing errors, side by side, with one sentence explaining the difference.

Next week: the robot is asked where it ended up, and told to fix it.